Cletus: Section 1, Day 1

> To copy code: select it here, or click a "Copy on GitHub" link and use the copy button there. Full note online: https://github.com/mangucletus/AI-Engineering-Course/blob/main/section-1/cletus-section1-day1.md

- Date: [YYYY-MM-DD]
- Course: Week 1 Day 1, running a first LLM locally, setting up the environment, and building a website summarizer
- Lab notebook: week1/day1.ipynb (the free Ollama version is in week1/solutions/day1_with_ollama.ipynb)
- Time spent: [e.g. 2h]

Key concepts

- LLM: a model that predicts the next bit of text (a token) based on the text before it.
- Ollama: lets you run open-source models on your own laptop for free. Use llama3.2, or llama3.2:1b if your machine is small. Skip llama3.3, at 70B it's way too big.
- Frontier models: the biggest closed models like GPT, Claude and Gemini. You reach them through an API and pay per use.
- API key and .env: the key goes in a file called exactly .env in the project folder and gets loaded with load_dotenv(). The .env file should never be committed.
- System prompt: tells the model what its job is and what tone to use.
- User prompt: the actual question or content we want a response to.
- Messages: the API expects a list of dicts, each with a role (system, user, and later assistant) and the content.
- Chat Completions API: we send the messages and the reply comes back in response.choices[0].message.content.
- Summarization is one of the most common business uses: news, reports, CVs, reviews, meeting notes.

Code / examples

One-time setup

Run these in a terminal: the Terminal app on Ubuntu, or PowerShell on Windows.

1. Install git and uv.

Linux (Ubuntu):

Copy on GitHub

```
sudo apt install -y git
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

Copy on GitHub

```
winget install --id Git.Git -e
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close the terminal and open a new one so the uv command is found, then check it works:

Copy on GitHub

```
uv --version
```

2. Get the course code and install its packages (same on Linux and Windows):

Copy on GitHub

```
git clone https://github.com/ed-donner/llm_engineering.git
cd llm_engineering
uv sync
```

3. Add your OpenAI key to .env inside the llm_engineering folder. Replace the placeholder with your real key. Creating the file by hand in Cursor or VS Code works too.

Linux (Ubuntu):

Copy on GitHub

```
echo "OPENAI_API_KEY=sk-proj-your-key-here" >> .env
```

Windows (PowerShell):

Copy on GitHub

```
Add-Content .env "OPENAI_API_KEY=sk-proj-your-key-here"
```

4. Install Ollama.

Linux (Ubuntu):

Copy on GitHub

```
curl -fsSL https://ollama.com/install.sh | sh
```

Windows (PowerShell), or download the installer from https://ollama.com/download:

Copy on GitHub

```
winget install --id Ollama.Ollama -e
```

Then chat with a local model (same on both). Type /bye to leave.

Copy on GitHub

```
ollama run llama3.2
```

On a smaller machine use the 1B version instead:

Copy on GitHub

```
ollama run llama3.2:1b
```

The website summarizer

Run this in a notebook cell inside the course's week1 folder, so the scraper import works. Pick the .venv kernel first.

Copy on GitHub

```
from dotenv import load_dotenv
from openai import OpenAI
# scraper.py is in week1; it returns a page's title and text (first 2,000 chars)
from scraper import fetch_website_contents

load_dotenv(override=True)
openai = OpenAI()  # picks up OPENAI_API_KEY from the environment

system_prompt = "You summarize websites in markdown, ignoring navigation text."
user_prompt_prefix = "Here are the contents of a website. Provide a short summary:\n\n"

def summarize(url):
    website = fetch_website_contents(url)
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt_prefix + website},
    ]
    response = openai.chat.completions.create(model="gpt-4.1-mini", messages=messages)
    return response.choices[0].message.content

print(summarize("https://edwarddonner.com"))
```

The flow is: scrape the page, build the messages, call the model, read the reply.

Exercise / challenge

1. Change the tone in the system prompt (reply in Spanish, be formal, be funny) and compare the summaries.
2. Build a summarizer without a website. Paste an email into the user prompt and ask for 3 bullet points and a subject line. Starter code (run it after the summarizer cell):

Copy on GitHub

```
system_prompt = "You summarize emails in 3 bullet points and suggest a subject line."
email = """
Paste the email text here
"""
messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": email},
]
response = openai.chat.completions.create(model="gpt-4.1-nano", messages=messages)
print(response.choices[0].message.content)
```

3. Try this one. It comes back almost empty. Why? (answer below)

Copy on GitHub

```
print(summarize("https://openai.com"))
```

Questions & blockers

Things to watch out for: - Sites built with JavaScript (React apps) or with bot protection return little or no text, because the scraper only reads the raw HTML. Selenium or Playwright can get around that. - A NameError in the notebook usually

means the cells weren't run from the top. Also pick the .venv kernel for each notebook. - OpenAI keys start with sk-proj-. Make sure there are no spaces around the key in .env. - The file must be called exactly .env, not .env.txt. On Windows, turn on "File name extensions" in File Explorer to check.

To discuss with my partner

- Where could we use summarization at work?
- What setup problems did each of us hit (Cursor, uv, kernel, API key)?

Resources

- Week 1 code: https://github.com/ed-donner/llm_engineering/tree/main/week1
- Setup guide: setup/SETUP-new.md, and setup/troubleshooting.ipynb if something breaks
- Using free models instead of paid APIs: guides/09_ai_apis_and_ollama.ipynb